# Woolworths Online Order and Delivery System – Enterprise Systems Analysis, Design, and Modelling Project-II

Shawn Lee (23204035)

# Table Of Content

# Summary of Systems Analysis

From the analysis of part 1, a proposal has been conducted for Woolworths New Zealand's Online and Delivery System. Based on some assumptions made. The previous report analyzed the identified key operational problems across 3 departments. These departments include store operations, supply chain & Logistics, and customer service.

The main issue found during the system analysis is that there is currently a heavy reliance on manual processes, which reduces real-time data visibility and efficiency in staff communication. This information is collected through multiple sources, which include the original problem statement, interviews with stakeholders, and, lastly, analyzing legacy systems. This directly helped produce the business goals, user requirements, functional requirements, and non-functional requirements.

In part 1, both the initial class diagram and use case diagram are created. The use case diagram includes both the customer-facing use case and another use case diagram for the internal management system. The initial class diagram covers classes related to customer, staff order, inventory, delivery, and reporting. Upon creating this, several weaknesses are found in the class diagram, including the absence of abstract classes and interfaces, and redundancies.

## Scope Refinement

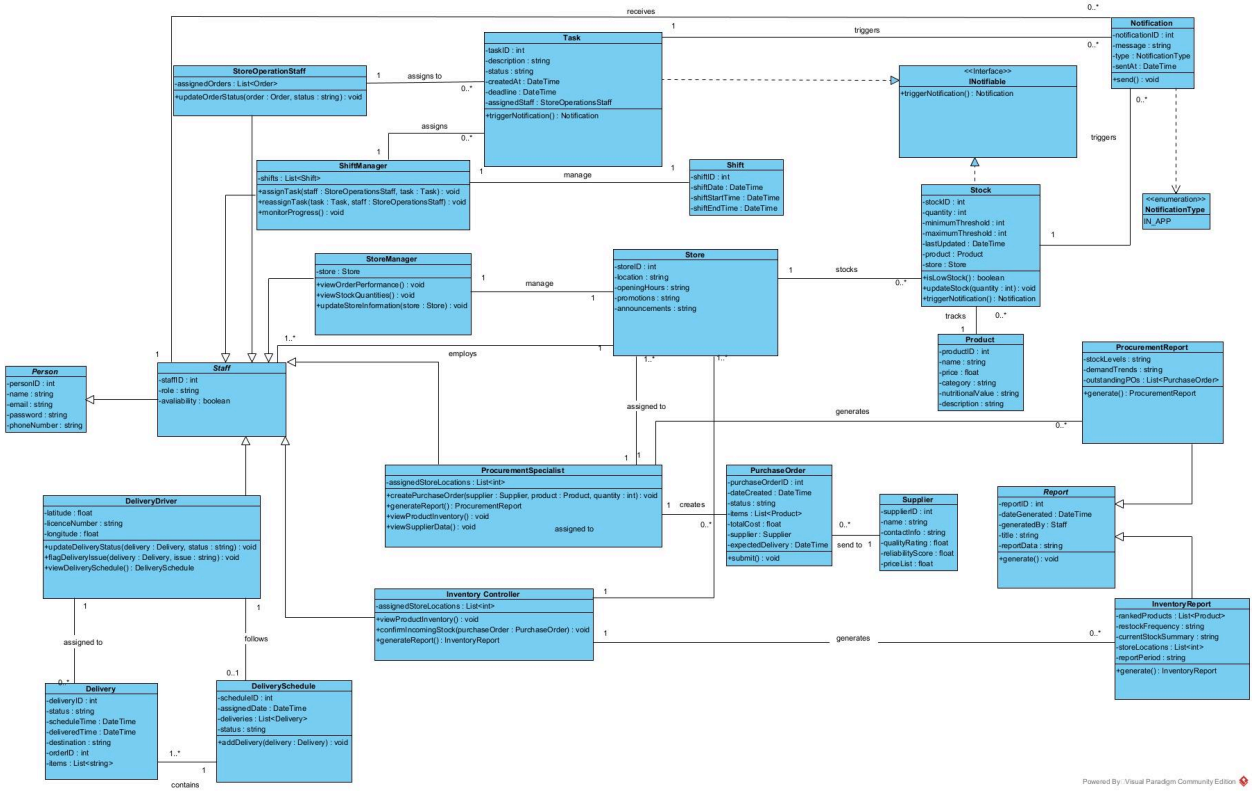
Continuing this report, the scope has been reduced to just focusing on the internal management system, and the customer-facing Online platform is out of the scope. Customer and customer service representative functionality has been excluded due to time constraints. Additionally, delivery management has been moved from the supply chain & logistics department towards the customer service department. The scope covered within the internal management system would be the Store operation staff, Shift manager, store manager, procurement specialist, inventory controller, and delivery driver.

Phase 2 will provide the refinement of the class diagram, introducing dynamic models through sequence and activity diagrams. Additionally, applying design patterns to improve system design. Lastly, compare initial designs to the final design and reflect on the choices made.

# Refined Static and Dynamic Modelling

## Refined Class Diagram

- ☑ ~~Refined UML Class Diagram with abstraction and interfaces~~




This updated iteration of the class diagram shows:

- **Abstract classes of Person, Staff, and Report (Italic)**

Since these classes are never initialized directly, Person and Staff are base classes for the subclasses of staff. Report is the base class for the Procurement report and the Inventory report class.

- **Enum NotificationType**

Changed from a string attribute called “type” to adding an enum “NotificationType”, even though there is just 1 constant value in NotificationType, it would be helpful for future different types of notifications, for example, email or SMS.

- **Changed currentLocation**

From the risk identified from phase 1, the currentLocation variable in the DeliveryDriver class is split into Longitude and latitude to enable exact distance calculations for delivery scheduling.

- **Added Interface**

An interface has been added to enforce methods on multiple classes. The interface called “INotifiable” is used for enforcing the triggerNotification method on both the Task and Stock classes.

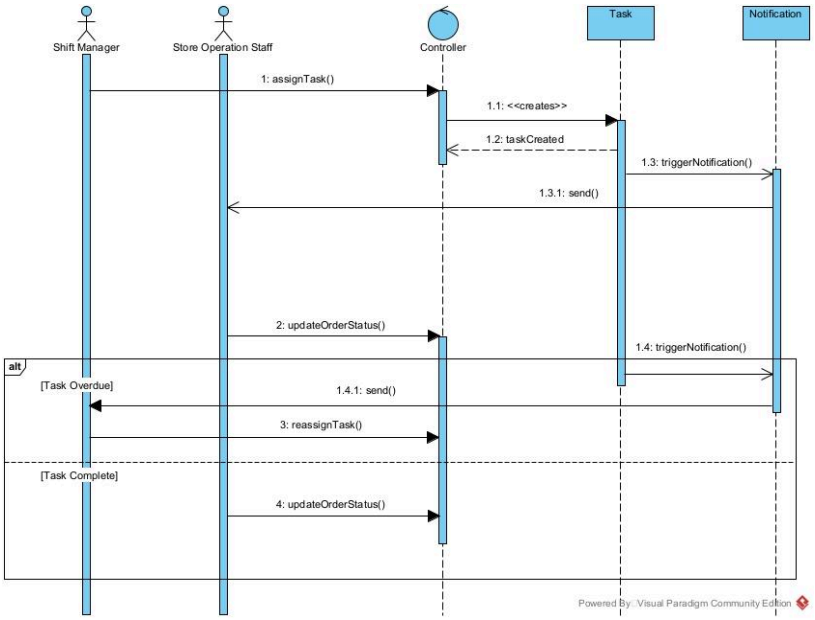
- **Scope Reduced**

As stated in the Scoper refinement, there will be no customer-facing functionalities, thus customer, cart, shoppingList, ShoppingListItem, Customer service representative, complaints, and refund.

## Sequence Diagram

- ☐ Minimum of three Sequence Diagrams

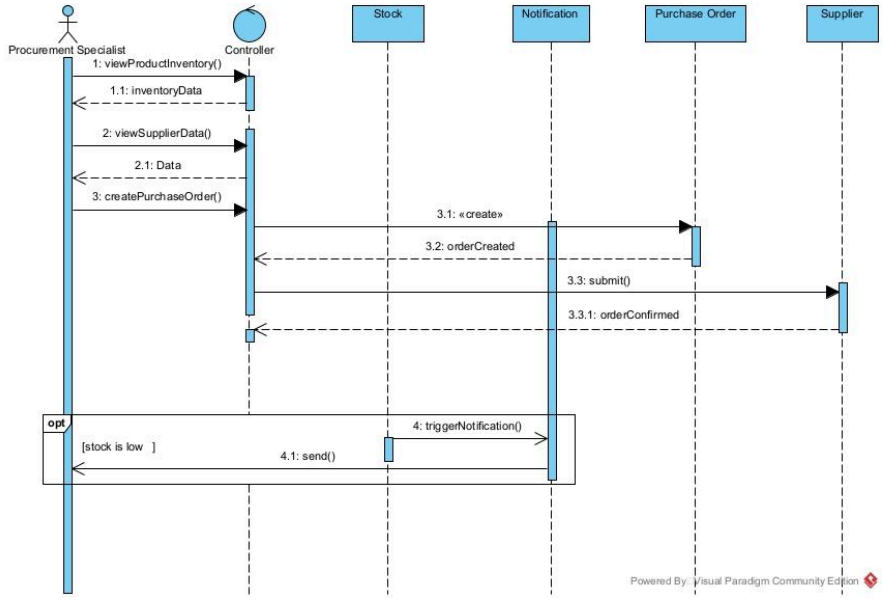
### Assign Task Sequence Diagram




This sequence diagram shows how the shift manager allocates tasks to the store operations staff for picking and packing. The process starts when the Shift Manager lifeline calls the assignTask() method, which sends a request to the controller lifeline, which initializes a task object. Once the task object is created, the task object calls the triggerNotification() method class asynchronously towards the Notification lifeline. This triggers the send() method, which is how Store Operation Staff is notified of a new task to complete.

The Store operation Staff calls the updateOrderStatus() method when there are changes to the pick and pack task. An alt combined fragment is used to show the alternate flow of whether the task is overdue or task is completed. The task overdue flow shows the Task lifeline calling the triggerNotification() method, which notifies the Shift Manager. The shift manager then calls the reassignTask() method. The task completion flow just shows calling the updatedOrderStatus() to change the status to completed.

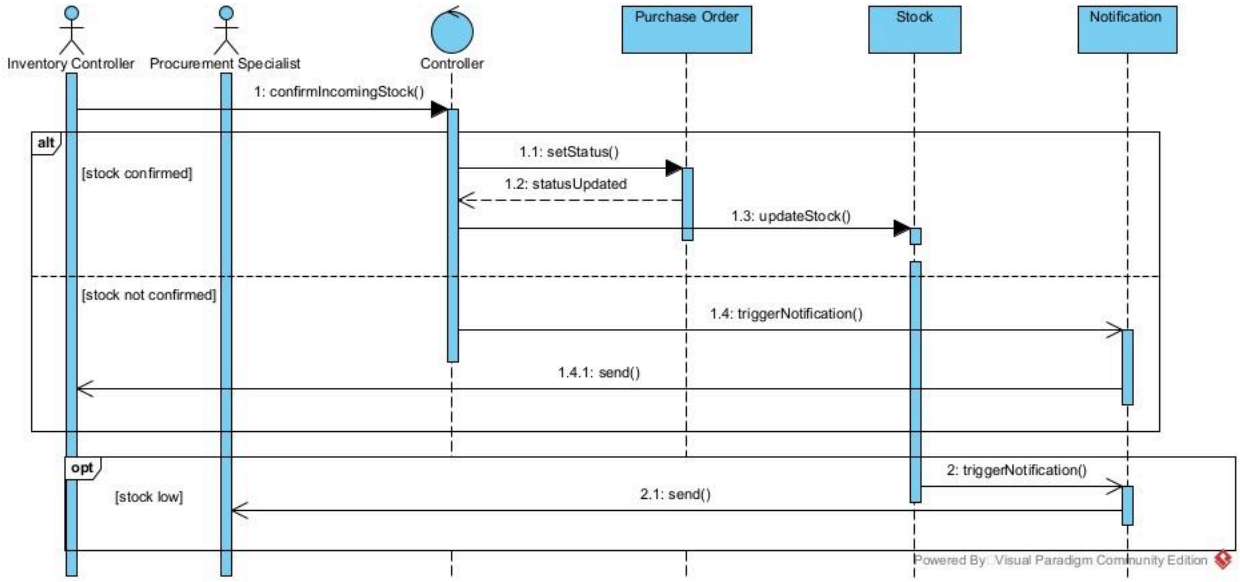
**Low Stock Notification and Purchase Order Sequence Diagram**




This sequence diagram shows the sequence of events occurring during the procurement specialist's purchasing of orders from suppliers. It begins with the Procurement Specialist calling the viewProductInventory() method towards the controller lifeline, which returns inventory data. Then the procurement specialist calls the viewSupplierData() method on the controller to return data to gain more info about which supplier to choose. Once a supplier is chosen, a new purchase order object is initialized, and the submit() method is called towards the supplier, which returns the order statement.

The opt combined fragment shown is the optional flow when the isLowStock() method returns true, which triggers the triggerNotification() method asynchronously. The send() method is called to notify the procurement specialist.

**Confirm Incoming Stock Sequence Diagram**




This sequence diagram depicts the process of an Inventory Controller checking incoming stock from a supplier shipment and immediately notifying the Procurement Specialist of the result. The sequence begins with the Inventory controller calling the confirmIncomingStock() method on the controller. An alternate combined fragment has 2 flows. The stock confirmed flow is when the purchase order is successfully completed, which then calls the setStatus() method, returns the status, and then calls the updateStock() method. The stock not confirmed flow calls the triggerNotification() method on the Notification object, which calls the send() method to notify the Inventory Controller.

As stated in the Low Stock Notification and Purchase Order Sequence Diagram explanation, the optional combined fragment is the same in the Confirm Incoming Stock Sequence Diagram.

## Activity Diagrams / State Machine Diagrams

- ☐ Minimum of 2 activity diagrams or state machine diagrams

# Design Pattern Application and Implementation

## Application

- ☐ Application of at least two GoF design patterns
- ☐ UML diagrams showing pattern structure

## Implementation

- ☐ Corresponding C# implementation demonstrating each pattern

# Comparative Design Analysis

Students must provide a critical comparison of the system design before and after the application of design patterns, addressing the following aspects:

- ☐ Changes to class structure and responsibility allocation
- ☐ Improvements in cohesion and reductions in coupling
- ☐ Impact on extensibility and maintainability
- ☐ Consequences for C# implementation complexity
- ☐ Examples of how new requirements can be accommodated more easily

The analysis must be supported by UML diagrams and concrete references to the C# codebase.

# Critical Reflection

## Design Pattern Tradeoffs

- [ ] Design trade-offs introduced by design patterns

## Limitations of the final design and implementation

- [ ] Limitations of the final design and implementation

## Lessons learned

- [ ] Lessons learned from aligning UML models with C# code.

# References

# Appendices

- GitHub (https://github.com/ShawnLeeyz/Woolworths-Systems-Analysis-Design-and-Modelling-Project-I.git)